

GRUCell#

<https://pytorch.org/docs/stable/generated/torch.nn.GRUCell.html>

Description:

A gated recurrent unit (GRU) cell. where σ is the sigmoid function, and $\odot$ is the Hadamard product. `input_size` (int) – The number of expected features in the input `x` `hidden_size` (int) – The number of features in the hidden state `h` `bias` (bool) – If False, then the layer does not use bias weights `b_ih` and `b_hh`. Default: True

Function Signature

```
class torch.nn.GRUCell(input_size, hidden_size, bias=True,
device=None, dtype=None)[source]#
```

Parameters

Inputs: `input`, `hidden`

Outputs: `h'`

Shape:

Code Examples

```
>>> rnn = nn.GRUCell(10, 20)
>>> input = torch.randn(6, 3, 10)
>>> hx = torch.randn(3, 20)
>>> output = []
>>> for i in range(6):
...     hx = rnn(input[i], hx)
...     output.append(hx)
```

Notes & Warnings

NOTE All the weights and biases are initialized from $U(-k, k)\sqrt{k}$ where $k = \frac{1}{\text{hidden_size}}$

Detailed Documentation

Documentation

A gated recurrent unit (GRU) cell.

where σ is the sigmoid function, and $\odot$ is the Hadamard product.

input_size (int) – The number of expected features in the input x

hidden_size (int) – The number of features in the hidden state h

bias (bool) – If False, then the layer does not use bias weights b_{ih} and b_{hh} . Default: True

input : tensor containing input features

hidden : tensor containing the initial hidden state for each element in the batch. Defaults to zero if not provided.

h' : tensor containing the next hidden state for each element in the batch

input: (N, H_{in}) or (H_{in}) tensor containing input features where $H_{in} = \text{input_size}$.

hidden: (N, H_{out}) or (H_{out}) tensor containing the initial hidden state where $H_{out} = \text{hidden_size}$. Defaults to zero if not provided.

output: (N, H_{out}) or (H_{out}) tensor containing the next hidden state.

weight_ih (torch.Tensor) – the learnable input-hidden weights, of shape $(3 \times \text{hidden_size}, \text{input_size})$

weight_hh (torch.Tensor) – the learnable hidden-hidden weights, of shape $(3 \times \text{hidden_size}, \text{hidden_size})$

bias_ih – the learnable input-hidden bias, of shape $(3 \times \text{hidden_size})$

bias_hh – the learnable hidden-hidden bias, of shape $(3 \times \text{hidden_size})$

All the weights and biases are initialized from $U(-k, k)$ where $k = \frac{1}{\sqrt{\text{hidden_size}}}$

On certain ROCm devices, when using float16 inputs this module will use different precision for backward.